CS6135 VLSI Physical Design Automation
Homework 3: Detailed Placement

賈俊佑 studentID:114062645

1.Introduction

Implement a detailed placer to minimize the total wirelength of a given placement. Given a legalized placement (including both movable modules and fixed modules) and the corresponding netlist, the detailed placer must keep the positions of all fixed modules unchanged while adjusting the positions of movable modules to reduce the overall wirelength.

2.Objective

Minimize the total wirelength of the placement by adjusting movable modules. The total wirelength W of a set N of nets is computed as:

W = $\sum_{ni\epsilon N} HPWL(n_i)$

$HPWL(n_i)$ represents the half-perimeter wirelength of net n_i, and special nets (listed in SPECIALNETS section in DEF file) are excluded from the calculation. To simplify wirelength calculation, each pin of a module m_i is located at the bottom-left coordinates of m_i. After performing detailed placement, the resulting placement must remain legalized, meaning:

1) No modules overlap.
2) The lower-left corner of each movable module must align exactly with its corresponding row and site.

3.Result

3.1Config

In all experiments, the detailed placer was run with a fixed configuration. The parameter values used in my implementation are summarized as follows:

- time_limit_sec = 250.0
  Time budget (in seconds) used by is_timeout() for the main detailed placement loop.
- reserve_for_shift = 10.0
  Reserved time (in seconds) for the final shift_components() stage; this value is subtracted from time_limit_sec when checking timeouts.
- batch_size = 512
  Number of cells processed per batch in both the independent-set matching (ISM) and global swap stages.
- ism_set_size = 64
  Maximum size of the independent set constructed around each ISM pivot. Each LAP instance is solved on at most 64 cells.
- ism_max_candidates = 128
  Upper bound on the number of candidate cells collected around each pivot before building the independent set.
- optimal_region_x_padding_factor = 5
  Padding factor used in find_optimal_region(). The x-range of the optimal region is expanded by 5 × cell_width on both sides before being clipped to the chip boundary.
- ism_range_factor = 5
  Horizontal search range for ISM candidates is set to
  range = cell_width × ism_range_factor,
  limiting candidate neighbors to those whose x-coordinates fall within this window.
- random_seed = 42
  Fixed random seed for the internal RNG, ensuring reproducible shuffling of components and batches. (If set to 0, a time-based seed is used instead.)

3.2 Final Result

```
+-----------------------------------------------+
|                                               |
|    This script is used for PDA HW3 grading.   |
|                                               |
+-----------------------------------------------+
host name: ic21
compiler version: g++ (GCC) 9.3.0

grading on 114062645:
 checking item          | status
------------------------|--------
 correct tar.gz         | yes
 correct file structure | yes
 have README            | yes
 have Makefile          | yes
 correct make clean     | yes
 correct make           | yes

  testcase |            hpwl |    runtime | status
-----------|-----------------|------------|--------
   public1 |        81615800 |       1.89 | success
   public2 |      3600475666 |      17.90 | success
   public3 |     29887768200 |     258.20 | success
   public4 |     41401084250 |     266.56 | success
+-----------------------------------------------+
|                                               |
|   Successfully write grades to HW3_grade.csv  |
|                                               |
+-----------------------------------------------+
```

4.Additional Elements Description (LEF and DEF)

Additional Elements from LEF (Library Exchange Format):

1. LAYER: Defines the physical fabrication layers (such as Metal or Via layers) and their associated design rules. It specifies essential properties like minimum width, minimum spacing, pitch, and the preferred routing direction (horizontal or vertical) for each layer.

2. VIA: Describes the geometric structure of the connections between different conducting layers. It defines the shapes (rectangles) and dimensions required to form a contact (cut) between two metal layers.

3. OBS (OBSTRUCTION): Specifies blockages within a macro or standard cell. These are regions where the router is prohibited from placing wires (usually to protect internal cell structures), ensuring that routing over the cell does not cause short circuits.

Additional Elements from DEF (Design Exchange Format):

1. TRACKS: Defines the routing grid for each metal layer. It specifies the starting coordinate, the number of tracks, and the step size (spacing) between tracks, providing the necessary grid information for the router to align wires.

2. SPECIALNETS: Describes the connectivity and physical geometry of special nets, most commonly Power (VDD) and Ground (VSS). Unlike regular signal nets, these are often routed with special widths or shapes (like stripes or rings) and are handled separately.

3. GCELLGRID: Defines the grid used for global routing. It specifies the starting coordinates, number of divisions, and the step size (spacing) for the global routing cells (GCells) in both X and Y directions, helping the router manage routing resources.

5.Method (Implementation)

5.1 Overall Workflow

The detailed placement algorithm is implemented as an iterative refinement process aimed at minimizing the total HPWL while maintaining strict legalization constraints. The core architecture of the placer is designed to exploit parallelism on a multithreaded CPU, adopting the Batch-Based Concurrent Detailed Placement framework proposed in ABCDPlace[1].

After an initial initialization phase (place_components), the algorithm enters a continuous optimization loop. Within each iteration, four distinct optimization strategies are executed sequentially to reduce the total HPWL:

1. Local Reordering: Permutes cells within a small sliding window (size 3 or 4) to resolve local entanglements.
2. Independent Set Matching (ISM): Solves the assignment problem for independent sets of cells to optimize positions globally within a local range.
3. Global Swap: Moves or swaps cells across different rows or distant

regions based on calculated optimal regions.
Finally, a shift_components pass is performed to compact cells into whitespace for fine-grained improvement. To maximize efficiency, OpenMP is utilized to parallelize these stages, ensuring that non-conflicting tasks are executed simultaneously.

The pseudo code is as follows
Algorithm: Iterative Detailed Placement Refinement

Input: Legalized Placement, Netlist
Output: Optimized Placement

// Map components to rows and sites based on coordinates
Initialize: place_components()

Repeat

    // Stage 1: Local Topology Correction (Coarse Window)
    LocalReorder(window_size = 4)

    // Stage 2: Global Optimization via Matching
    IndependentSetMatching()

    // Stage 3: Global Movement
    GlobalSwap()

    // Stage 4: Local Topology Correction (Fine Window)
    LocalReorder(window_size = 3)

Until (No Improvement in Total HPWL) OR (Time Limit Exceeded)

// Stage 5: Final Fine-Tuning
ShiftComponents()

5.2 Detailed Strategies

5.2.1 Concurrent Independent Set Matching (ISM)

This stage is designed to explore solution spaces involving multiple cells

simultaneously. Following the methodology in ABCDPlace[1], we implement a Batch-Based approach to enable concurrency on the CPU. The process works as follows:

- Batch Selection: Instead of processing the entire netlist sequentially or constructing a single massive independent set, the algorithm iterates through randomized batches of "pivot" components (defined by config_.batch_size). This allows for better cache locality and easier parallelization.
- Candidate Selection & Independent Set Extraction: For each pivot in a batch, the algorithm searches for neighbor candidates within a defined range (ism_range_factor). To ensure thread safety and validity during concurrent execution, we construct an Independent Set where no two chosen cells share a common net. This independence guarantees that moving one cell does not affect the HPWL calculation of another in the same set.
- Bipartite Matching with Hungarian Solver (Contrast with Ref [1]): A significant distinction from the ABCDPlace[1], which employs an Auction Algorithm optimized for massive GPU parallelism, is our adoption of the Hungarian Algorithm. We model the placement optimization within an independent set as a Minimum Weight Bipartite Matching problem. To solve this, we implemented a custom HungarianSolver class.
    - Why Hungarian? While the Auction algorithm is generally preferred for distributed or GPU-based solving due to its parallel nature, the Hungarian algorithm (O(n^3)) guarantees finding the exact global minimum cost assignment.
    - Implementation: We calculate a cost matrix representing the HPWL change for every possible move. The HungarianSolver::solve() function is then instantiated for each independent set to determine the optimal assignment. Given that our independent sets are constrained to a manageable size (e.g., 64 cells), this exact solver provides superior local quality without becoming a runtime bottleneck on the CPU.
- Parallel Execution: The solving of these independent sets is parallelized using OpenMP (#pragma omp parallel for), allowing multiple local matching problems (each running its own Hungarian

solver instance) to be solved simultaneously across different threads.

5.2.2 Concurrent Global Swap

This strategy performs coarser-grained optimization by attempting to move cells to their "Optimal Regions".

- Optimal Region Calculation: For every movable component, an optimal rectangular region is calculated based on the median coordinates of all connected nets.
- Batch-Based Search: Similar to ISM, cells are processed in batches. For each cell, the algorithm searches its optimal region (which may span multiple rows) for the best location. It evaluates two types of moves: moving into whitespace or swapping with another component.
- Parallelism & Conflict Resolution: The search for the "Best Move" is fully parallelized. Each thread calculates the potential HPWL reduction for a cell. However, applying these moves requires care to prevent conflicts (e.g., two cells swapping with the same target). Therefore, we employ a two-phase approach: a parallel search phase followed by a sequential application phase that only executes non-conflicting moves.

5.2.3 Concurrent Local Reordering

Local reordering handles detailed permutations within rows to untangle local connections. To parallelize this inherently sequential sliding-window process, we adopt an Independent Row Group strategy.

- Row Dependency Analysis: We construct a dependency graph where rows connected by nets are considered "adjacent".
- Graph Coloring: Using a graph coloring algorithm, we group rows into independent sets. Rows within the same set do not share any nets.
- Parallel Execution: Rows in the same independent group are processed in parallel. Within each row, a sliding window enumerates all permutations (e.g., 3! = 6) of adjacent cells and applies the order that minimizes HPWL.

5.2.4 Component Shift

The final stage of the optimization pipeline acts as a greedy heuristic for fine-grained tuning. This step focuses on utilizing the available whitespace within rows to compact the placement.

- Greedy Shifting: The algorithm iterates through all movable components and attempts to shift them one site to the left or right into adjacent empty space.
- Evaluation & Acceptance: For each potential shift, the HPWL is recalculated. The move is accepted if and only if it results in a strict reduction of the total wirelength.
- Legalization: The shift respects the boundaries of fixed modules and other components, ensuring that the placement remains strictly legalized (no overlap) at all times. This effectively utilizes small gaps between modules without altering their relative ordering, correcting minor placement drifts that may have occurred during the global swapping phases.

6.Conclusion: Learnings and Challenges

In this homework, I implemented a detailed placer to minimize the total wirelength (HPWL) of a given legalized placement. My implementation strategy was primarily based on the Batch-Based Concurrent Detailed Placement framework proposed in ABCDPlace[1].

The most significant challenge I encountered was adapting the algorithms designed for massive GPU parallelism in [1] to a multithreaded CPU environment. Specifically, the Auction Algorithm used in the literature for solving matching problems is optimized for GPU throughput but can suffer from convergence issues and approximation errors.

Adopting the Hungarian Algorithm: To overcome this, I deviated from the paper and implemented the Hungarian Algorithm for the Independent Set Matching (ISM) stage. Although theoretically more computationally intensive (O(n^3)), it guarantees finding the exact optimal assignment for local problems. In my implementation, by keeping the independent set sizes manageable (e.g., 64 cells), I found that the Hungarian solver provided superior solution quality (HPWL reduction) compared to approximate methods, without becoming a runtime bottleneck.

Besides the robust matching strategy, two specific implementation choices significantly contributed to the placer's performance:

1. Optimal Region-Based Global Swap: Instead of random or purely greedy swaps, I implemented a targeted Global Swap strategy. By calculating the "Optimal Region" for each cell based on its nets' median positions, the algorithm efficiently narrows down the search space. The two-phase conflict resolution (parallel search followed by sequential application) ensured that large-scale movements could be performed concurrently without causing illegal overlaps, effectively untangling global connections.
2. Component Shift: The final inclusion of a greedy Component Shift pass proved essential for fine-tuning. By utilizing the fragmentation (whitespace) within rows, this technique compacted the placement and squeezed out the remaining wirelength reductions that were too small for the global swap to catch.

In conclusion, this assignment taught me that adapting state-of-the-art algorithms requires careful consideration of the target hardware and problem constraints. By replacing the GPU-centric Auction algorithm with the CPU-friendly, exact Hungarian algorithm, and implementing a robust, overlap-aware Global Swap, I was able to achieve a stable and high-quality detailed placer. The experience highlighted that exact local solutions (like Hungarian) combined with broad global heuristics (like Optimal Region Swap) often yield the best balance between performance and quality in detailed placement.

7.Declaration on the Use of AI Tools

In this project, I used AI tools including Google's Gemini and OpenAI's ChatGPT, for two primary purposes:

1. Debugging: I submitted segments of my C++ code to the AI to assist in identifying logical errors and syntax issues. This was particularly helpful for tracing segmentation faults and resolving complex boundary conditions within the placement grid that were difficult to debug manually.

2. Code Improvement and Optimization: I consulted the AI for suggestions on how to simplify specific logic blocks within my algorithm, such as the implementation of the horizontal swap and Component Shifting functions. I sought recommendations on how to refine the code structure to ensure the placement remained legal without compromising the execution speed.

8.References

[1] Y. Lin, W. Li, J. Gu, H. Ren, B. Khailany and D. Z. Pan, “ABCDPlace: Accelerated Batch-Based Concurrent Detailed Placement on Multithreaded CPUs and GPUs,” in IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, vol. 39, no. 12, pp. 5083-5096, 2020.

[2] M. Pan, N. Viswanathan and C. Chu, “An Efficient and Effective Detailed Placement Algorithm,” in Proceedings of International Conference on Computer-Aided Design, 2005.